GAN-GUIDED KINODYNAMIC MOTION PLANNING

A Learning-Based Generative Sampler for Basic RRT

Saketh Tammisetti
SCC Lab | UF Mechanical & Aerospace Engineering
PI: Dr. Rushikesh Kamalapurkar
UF SSTP 2026

INTRODUCTION

MOTION PLANNING CONTEXT
Motion planning moves a system from a start state to a goal region without hitting obstacles.
Kinodynamic planning adds velocity, acceleration, control bounds, and dynamics.
Sampling-based planners grow a tree by sampling and attempting local motions.
RRTs spread that tree quickly into unexplored regions.

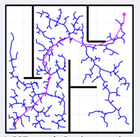


Figure 1. RRT growth. Random samples create a branching tree that spreads through the available state space.

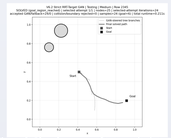


Figure 2. Planning scene. Start, goal, obstacle constraints, sampled branches, and a final feasible route.

HOW RRT BUILDS A PLAN
Sample, find the nearest node, extend, repeat until the goal connects.
KINODYNAMIC PROBLEM
State x = [p_x, p_y, v_x, v_y]. Inputs: start, goal, obstacles, dynamics, bounds, controls.
VALID SOLUTION
Reach the goal, stay in bounds, avoid obstacles, satisfy the motion model.
MODELING GAP
Geometric steering ignores velocity. LQR–CBF steering is safe but too slow to repeat. A learned replacement must preserve exploration and defer to explicit safety checks.

RESEARCH OBJECTIVES

THE PROBLEM
RRT explores well, but geometric steering ignores velocity. An LQR–CBF expert fixes that but costs an optimization per expansion.

RESEARCH QUESTION
Can a conditional GAN learn the expert steering transition while the planner keeps random sampling, nearest-parent selection, one-child growth, and independent safety checks?

OUR CLAIM
A generator trained on LQR–CBF transitions gives useful one-step proposals and high solve rates on unseen environments, without replacing the RRT core or safety filter.

DESIGN CRITERIA
- **RRT fidelity:** one sample, its nearest parent, at most one child.
- **Dynamics:** predict Δp and v_k+1, rebuild x_k+1, reject invalid motion.
- **Safety:** collision and boundary checks stay outside the network.
- **Generalization:** split by environment, freeze testing until model selection ends.

Controlled comparison: same scenarios and budgets for Basic RRT and GAN-guided RRT.

PRIMARY ENDPOINT
Valid-nontrivial solve rate on the frozen 300-case benchmark, read with accepted-edge collision violations.
SECONDARY ENDPOINTS
- **Prediction quality:** full-state, position, and velocity accuracy on held-out transitions.
- **Planner behavior:** runtime, accepted nodes, iterations, path cost, GAN share.
- **Baseline:** paired Basic RRT on the same 300 cases.

REAL WORLD IMPACT

Autonomous vehicles: plans motion that respects momentum, not just geometry.
Mobile robots: less wasted search in cluttered warehouses and indoor spaces.
Safety architecture: known dynamics and collision checks decide whether a proposal is accepted.
Computational value: better steering cuts expansions that miss the target or violate the model.
Scientific value: learned prediction stays separate from known physics, keeping error auditable.
Benchmark value: environment-disjoint splits reduce memorization of obstacle layouts.



METHODOLOGY

Figure 3. Full Training-to-Deployment Methodology

1. GENERATE EXPERT DATA — Create safe training targets.
→ **DEFINE CONDITION** — State, RRT sample, bounds, and obstacles.
→ **LQR PROPOSAL** — Compute target-directed control.
→ **CBF CORRECTION** — Project control into the safe admissible set.
→ **STORE LABELS** — Save [Δp, v_{k+1}]. 60k steps; 300 environments.

2. TRAIN AND VALIDATE — Learn without layout leakage.
→ **ENVIRONMENT SPLIT** — 240 / 30 / 30 worlds; 48k / 6k / 6k rows.
→ **BUILD FEATURES** — Normalize state and sample; encode relative obstacles.
→ **TRAIN GAN** — Fit G/D using prediction, dynamics, and adversarial loss.
→ **SELECT CHECKPOINT** — 240 + 10 epochs; batch 512; validate every 5.

3. DEPLOY STEERING — Freeze one repeatable RRT rule.
→ **SET CONDITION** — Parent, sample, obstacles, z = 0.
→ **RUN GENERATOR** — Shared encoder; separate position and velocity heads.
→ **REBUILD STATE** — Predict Δp and v_{k+1}; reconstruct x_{k+1}.
→ **CHECK SAFETY** — Reject invalid, out-of-bounds, or colliding edges.

4. TEST INSIDE RRT — Evaluate complete paths, not only one-step error.
→ **SAMPLE** — Draw one collision-free target; 18% goal bias.
→ **NEAREST PARENT** — Choose the nearest existing tree state.
→ **LEARNED STEER** — Generate one motion; apply known dynamics.
→ **TREE UPDATE** — Insert one child; test the goal.
→ **TEST** — 300 cases: solve, time, nodes, cost, collisions.

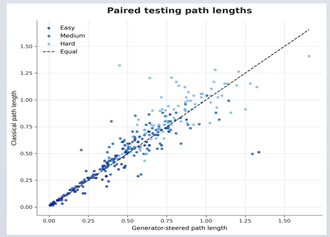
RRT PARENT SELECTION
$x_{near} = \arg\min_{x_i \in T} \|p_i - p_{rand}\|_2$
The exact random sample and nearest-parent rule are preserved. Learning changes only the local steering decision.

LQR–CBF EXPERT
$u^* = \arg\min_u \|u - u_{LQR}\|^2$ *subject to* $\dot{h}(x,u) + \alpha h(x) \geq 0$
LQR supplies target direction; the barrier constraint changes the action only when safety requires it.

LEARNED STATE TRANSITION
$\hat{y} = G_\theta(c,0) = [\Delta p, v_{k+1}]$
$p_{k+1} = p_k + \Delta p$
Using z = 0 creates one repeatable deployment output. Known bounds and continuous collision checks remain external to the network.

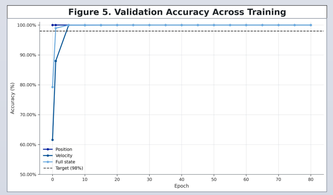
WHY THIS METHODOLOGY
Expert supervision provides useful and safe local transitions. Environment-level splitting tests generalization. The exact RRT sample preserves exploration. Deterministic deployment makes the reported model reproducible. State reconstruction, bounds, and continuous collision checks prevent the neural network from independently declaring an edge safe. Full rollouts test whether local predictions produce complete paths.

RESULTS & ANALYSIS



Interpretation: : Most cases lie near the equality line, indicating similar path lengths, but the slight upward bias shows that generator-steered paths were shorter overall

Paired metric — same 300 test cases	Basic RRT	GAN-guided RRT
Solve rate	99.00%	99.00%
Mean path cost on jointly solved cases	0.5813	0.5639
Mean runtime per case	0.1147 s	5.8215 s



WHAT FIGURE 5 MEASURES
Validation accuracy was evaluated using deterministic z=0. Position and velocity predictions were correct below 0.05 error, while full-state predictions were correct below 0.10 error.
RESULT
At epoch 30, validation position, velocity, and full-state accuracy all reached 100.00%. Testing accuracy remained 100.00%, 99.98%, and 100.00%, respectively.
WHY IT MATTERS
Every validation metric exceeded the 98% deployment target. Only one of 6,000 testing transitions exceeded the stricter velocity threshold.

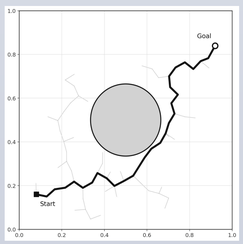


Figure 7. Easy case. The final route is short while unsafe branches near the obstacle are rejected.

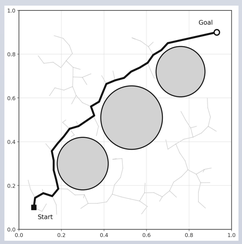


Figure 8. Medium case. The tree explores around obstacle constraints before extracting a feasible route.

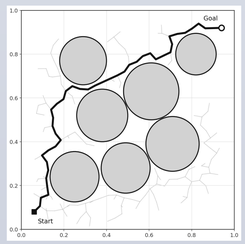
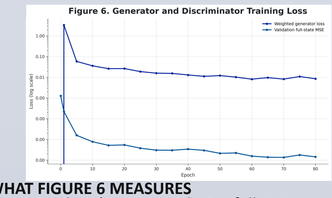


Figure 9. Hard case. Multiple obstacles force a curved detour before the goal connection is accepted.



WHAT FIGURE 6 MEASURES
Generator loss (position, velocity, full-state, control, dynamics-consistency); discriminator separability of generated vs. expert transitions.
RESULT
Generator loss: 3.3373 → 0.0159. Final discriminator: 50.00% accuracy, 0.6931 BCE.
INTERPRETATION
Prediction error fell as validation accuracy peaked. The 50% is real-vs-generated classification, not motion accuracy.

STATISTICAL ANALYSIS

STATISTICAL METHOD
Point estimate: $\hat{p}$ = 298 / 300 = 99.33%.
Uncertainty: resample the 30 testing environments with replacement 2,000 times, recompute solve rate, report the 2.5th and 97.5th percentiles.
Why cluster by environment: cases sharing an obstacle layout are correlated, so resampling rows would overstate independence.

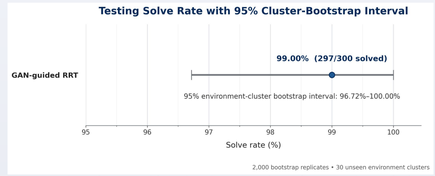


Figure 10. Testing solve rate with 95% environment-cluster bootstrap interval. The interval is 96.72%–100.00%.

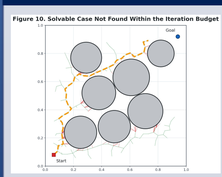
INTERPRETATION
The lower bound stays above 97%, supporting high success on unseen environments. This is planner-level uncertainty only; the separate 72.93% velocity-accuracy limitation still stands.

Table 1. Frozen testing statistic	Result
Eligible nontrivial cases	300
Solved cases	297
Solve rate	99.00%
Cluster-bootstrap 95% interval	96.72%–100.00%
Bootstrap replicates / environment clusters	2,000 / 30
Accepted-edge collision violations	0

CONCLUSIONS

RRT structure retained: Random sampling, nearest-parent selection, and one-child tree growth remained unchanged.
Learned component isolated: The generator replaced only the local one-step steering calculation.
Prediction performance: Testing position and full-state accuracy reached 100.00%; velocity accuracy reached 99.98%.
Planner performance: The system solved 297 of 300 unseen cases with zero accepted-edge collision violations.
Computational behavior: Planning averaged 5.8215 seconds, 140.32 accepted nodes, and 194.49 iterations.
Primary limitation: Neural inference and candidate evaluation still add substantial CPU runtime.
Overall conclusion: Learned steering supported accurate, dynamics-consistent planning while explicit feasibility checks preserved accepted-edge safety
TAKEAWAY:
- A 99.00% solve rate, 100.00% full-state accuracy, and zero accepted-edge collisions demonstrate strong simulated performance
- Runtime and real-world validation remain the primary engineering targets.

FUTURE WORK & LIMITATIONS



Limitations
- Velocity prediction stays less accurate than position prediction - the primary performance limitation.
- Full-state prediction is not yet exact.

Future Work
- Improve velocity prediction with dynamics-focused training.
- Test the planner on a real nonholonomic vehicle with real sensor and trajectory data.

KEY REFERENCES

[1] S. M. LaValle, "Rapidly-exploring random trees: A new tool for path planning," Dept. Comput. Sci., Iowa State Univ., Ames, IA, USA, Tech. Rep. TR 98-11, Oct. 1998.

[2] S. M. LaValle and J. J. Kuffner, Jr., "Randomized kinodynamic planning," Int. J. Robot. Res., vol. 20, no. 5, pp. 378–400, May 2001.

[3] I. J. Goodfellow et al., "Generative adversarial nets," in Advances in Neural Information Processing Systems 27, 2014, pp. 2672–2680.

[4] B. Ichter, J. Harrison, and M. Pavone, "Learning sampling distributions for robot motion planning," in Proc. IEEE Int. Conf. Robot. Autom. (ICRA), Brisbane, QLD, Australia, 2018, pp. 7087–7094.

[5] B. D. O. Anderson and J. B. Moore, Optimal Control: Linear Quadratic Methods. Englewood Cliffs, NJ, USA: Prentice-Hall, 1990.

[6] A. D. Ames, X. Xu, J. W. Grizzle, and P. Tabuada, "Control barrier function based quadratic programs for safety critical systems," IEEE Trans. Autom. Control, vol. 62, no. 8, pp. 3861–3876, Aug. 2017.